

Prouver l'exécution d'un automate (STARK)

Mathéo Le Dévéhat

23112

2025-2026

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

Problématique

Comment certifier qu'un texte confidentiel satisfait les propriétés d'un automate fini sans en divulguer le contenu ?

- Cas d'étude: connaissance d'un mot reconnu par un automate
- Enjeu: preuve courte, vérification rapide
- Motivations: Prouver son identité sans rien révéler de plus

Objectif

- Automate A connu publiquement
- Prouver: connaître un mot w reconnu par A
- Sans divulguer w

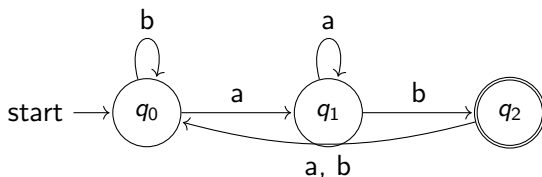


Figure: Un automate A : états, transitions, état initial/acceptant

- 1 Objectif et motivations
- 2 Prérequis**
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

Prérequis: Fonction de hachage

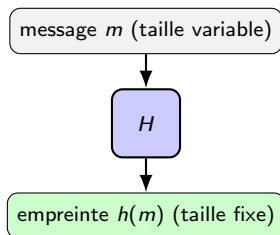
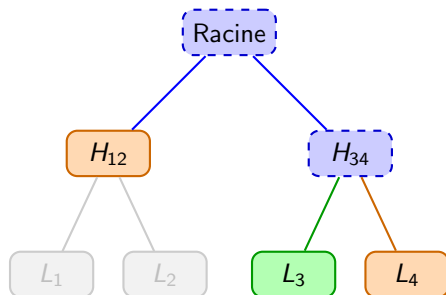


Figure: Une fonction de hachage: entrée variable, sortie fixe

Critères essentiels:

- ❶ Résistance aux collisions
- ❷ Résistance à la préimage
- ❸ Résistance à la deuxième préimage
- ❹ Distribution uniforme

Prérequis: Arbres de Merkle



- Preuve d'appartenance: **nœuds frères** seulement
- Recalcul itératif (**bleu**) jusqu'à la racine
- Taille de preuve: $\mathcal{O}(\log_2 N)$

Reconstruction

$$H_{\text{parent}} = \text{Hash}(H_g \parallel H_d)$$

Prérequis: L'heuristique de Fiat-Shamir

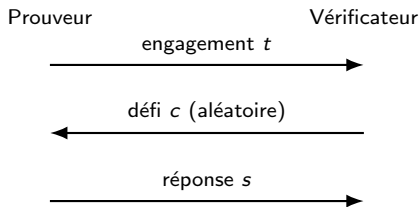


Figure: Version interactive

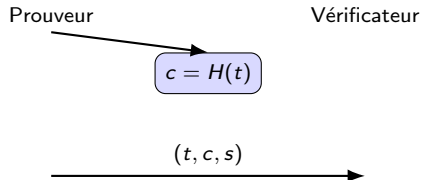


Figure: Version non interactive: H remplace le vérificateur

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions**
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

- **Prouveur**: forte puissance de calcul, données privées
- **Vérificateur**: accès public, faible puissance de calcul
- Cas d'usage: vérification de programmes sur une blockchain publique

Prouver l'exécution d'un automate

Déclaration naïve

Je connais un mot W accepté par un automate A

- Majuscules: variables publiques
- Minuscules: connues du seul prouveur
- Idée: prouver que chaque transition est correcte

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR**
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion

AIR - Algebraic Intermediate Representation

- AIR: transformer un calcul en problème algébrique
- But: prouver la validité de chaque transition

étape	état	caractère
0	1	B
1	4	B
2	1	A
3	2	B
4	4	A
5	2	ϵ

Table: Table de transitions

- Un polynôme par colonne (P_i, P_e, P_c) , via interpolation de Lagrange

$$P_e = \frac{11x^5}{120} - \frac{35x^4}{24} + \frac{65x^3}{8} - \frac{445x^2}{24} + \frac{887x}{60} + 1$$

3 contraintes à respecter:

- 1 L'exécution démarre à l'état initial
- 2 Chaque transition est valide (est dans la définition de l'automate donné en paramètre)
- 3 L'exécution termine sur un état final

Contraintes $\rightarrow$ polynômes, via:

$$P(x) = 0 \iff (X - x) \mid P \iff \frac{P}{X - x} \text{ est un polynôme}$$

$$\textcircled{1} \quad C_0 = \frac{P_e(0) - q_0}{X}$$

$$\textcircled{2} \quad C_1 = \frac{P_e(X+1) - P_t(P_e(X), P_c(X))}{\prod_{0 \leq i \leq n-1} (X - i)}$$

$$\textcircled{3} \quad C_2 = \frac{\prod_{q \in F} P_e(n-1) - q}{X - (n-1)}$$

P_t : polynôme de transition (état, caractère) $\rightarrow$ état suivant

Combiner les contraintes en un seul polynôme, pour appliquer FRI:

$$CP = \alpha_0 C_0 + \alpha_1 C_1 + \alpha_2 C_2$$

- $\alpha_0, \alpha_1, \alpha_2$: tirés aléatoirement par le vérificateur
- Garantissent $CP = 0 \iff C_0 = C_1 = C_2 = 0$ (en probabilités), même si le vérificateur choisit les coefficients

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI**
- 6 Récapitulatif
- 7 Conclusion

Protocole FRI: Fast Reed-Solomon Interactive Oracle Proof of Proximity

- But: montrer que CP s'annule sur toutes les étapes de calcul
- Solution naïve: envoyer le polynôme et tous les points
- Problème: aucune compression, aucune confidentialité

Objectif

Montrer que CP est un polynôme, en le ramenant degré par degré ($D \rightarrow D/2 \rightarrow \dots$) jusqu'à un polynôme constant.

$$\Delta(f, P) = \frac{|\{x \in D \mid f(x) \neq P(x)\}|}{|D|}$$

- On montre que f est δ -proche d'un polynôme: $P(\Delta(f, P) < \delta) \approx 1$
- L'opérateur FRI divise le degré par 2 sans perdre cette propriété
- Le vérificateur n'a plus qu'à vérifier des points alignés

Comment fonctionne FRI

Inspiration

Même idée que la FFT pour les polynômes.

- 1 On sépare le polynôme en un polynôme avec les puissances paires, et un autre avec les impaires $P(X) = p(X^2) + Xi(X^2)$
- 2 On tire un β aléatoirement (Fiat-Shamir)
- 3 On considère maintenant $\tilde{P} = p(X) + \beta i(X)$

Commitment

Il faut attester chacune des applications de l'opérateur FRI.

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif**
- 7 Conclusion

Voici ce que connaît le vérificateur:

- 1 l'engagement de CP
- 2 l'engagement de chaque application de FRI
- 3 tous les tirages aléatoires

- 1 Générer la trace de l'exécution de l'automate et de la fonction de hachage
- 2 Créer tous les polynômes de représentation intermédiaires
- 3 Évaluer ces polynômes et les engager
- 4 Créer CP (donc vérifier les contraintes)
- 5 Appliquer FRI sur CP itérativement
- 6 Répéter la dernière étape autant de fois qu'on veut (nombre de points d'évaluation)
- 7 Envoyer des métadonnées, les engagements, preuves d'engagements au vérificateur

Idée

L'idée est de refaire le processus inverse qu'à fait le prouver mais au lieu de calculer les polynômes on vérifie que les évaluations correspondent.

- ❶ Récupérer les métadonnées, engagement et preuves d'engagements
- ❷ Pour chaque point d'évaluation
 - ❶ Vérifier l'engagement pour l'état, le caractère, et la transition
 - ❷ Vérifier les engagements du hash
 - ❸ Reconstruire **l'évaluation** de CP
 - ❹ Pour chaque application de FRI
 - ❶ Vérifier les engagements
 - ❷ Appliquer FRI au point
 - ❸ Vérifier que le résultat est correct

Prouver l'exécution d'un automate

Déclaration correcte

Je connais un mot w accepté par un automate A tel que $h(w) = H$

- A, H : publics — w : secret du prouveur

Poséidon: une fonction de hachage "ZK-friendly"

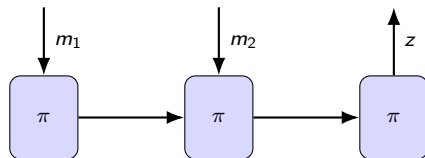


Figure: Éponge: absorption (m_i) puis extraction (z), via π

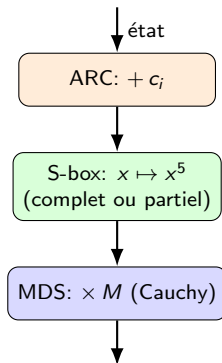


Figure: Un tour de la permutation π (répété $r_f + r_p$ fois)

Merci de votre attention

Sommaire

- 1 Objectif et motivations
- 2 Prérequis
- 3 Définitions
- 4 AIR
- 5 FRI
- 6 Récapitulatif
- 7 Conclusion**

- ❶ 01-02/25: Lecture des papiers sur Cairo, STARK et FRI
- ❷ 03/25: STARK 101
- ❸ 04-06/25: Polynômes et utilitaires en Ocaml
- ❹ 09/25: Idée AFD et premier prototype d'AIR
- ❺ 11/25: Prototype fini sans ZK
- ❻ 01/26: Fini la rédaction d'articles sur ce premier prototype
- ❼ 02/26: Fini prototype avec le hash
- ❽ 02/26: Gros problème

Annexe: Code – Poséidon (1/2)

```
1 class Sponge():
2     def __init__(self, rate, capacity, permutation, r_f, r_p, rc):
3         self.r = rate
4         self.c = capacity
5         self.pi = permutation
6         self.state = None
7         self.r_f = r_f
8         self.r_p = r_p
9         self.mds = generate_cauchy_matrix(rate+capacity)
10        self.rc = ARC
11        self.trace = []
12
13    def apply(self, data):
14        self.state = np.array([FieldElement(0) for i in range(0, self.c+self.r)])
15        self.trace = [self.state]
16        #remaining = self.pad(remaining)
17        chunks = [data[self.r*i:self.r*(i+1)] for i in range(0, len(data)//self.r)]
18        i = 0
19        while(len(chunks) > 0):
20            i+=1
21            self.state, trace = self.pi(self.state + np.pad(chunks[0], (0, self.c), mode='constant'), self
                .r_f, self.r_p, self.mds, self.rc)
22            chunks = chunks[1:]
23            self.trace.extend(trace)
24        return self.state[0:self.c], self.trace
25
26
27    def generate_cauchy_matrix(t):
28        x = [FieldElement(e) for e in range(0, t)]
29        y = [FieldElement(e) for e in range(t, 2*t)]
30        return np.array([[FieldElement(1)/(x[i]-y[j]) for j in range(0, len(y))] for i in range(0, len(x))])
31
32    def arc(state, rc):
33        return np.array(state) + rc
34
35    def mix(state, mds):
36        return mds @ np.array(state)
37
38    def full_s_box(state):
39        return np.array(state) ** 5
```

Annexe: Code – Poséidon (2/2)

```
1 def partial_s_box(state):
2     return [state[0] ** 5, *state[1:]]
3
4 def hades_round_permutation(state, r_f, r_p, mds, rc):
5     trace = []
6     for r in range(0, r_f+r_p):
7         state = arc(state, rc[r])
8         if (r_f/2 <= r < r_f/2+r_p):
9             state = partial_s_box(state)
10        else:
11            state = full_s_box(state)
12            state = mix(state, mds)
13            trace.append(state)
14    return state, trace
15
16 class Poseidon():
17     def __init__(self):
18         self.sponge = Sponge(7,1,hades_round_permutation,8,57, ARC)
19
20     def hash(self, data):
21         return self.sponge.apply(data)
```


Annexe: Code – Prouveur (1/7)

```
1 #word = open("text.txt", "r").read()
2 word = "I think STARKS are fun to learn"
3
4 import pandas as pd
5 from itertools import product
6 import time
7 from poseidon import Poseidon, ARC, generate_cauchy_matrix
8 from field import FieldElement
9 import numpy as np
10 from polynomial import Polynomial, interpolate_poly
11 from channel import Channel
12 from merkle import MerkleTree, verify_decommitment
13 from functools import reduce
14 from constants import N_QUERY, r_f, r_p, blowup_factor, T
15 import constants
16 from utils import Automata, primes
17 from tqdm import tqdm
18
19 start_all = time.time()
20
21 automata = Automata.from_json(open("automata.json", "r").read())
22 trace = automata.generate_trace(word)
23
24 mds = generate_cauchy_matrix(T)
25
26 hash_input = [FieldElement(x) for x in np.pad([automata.character_map[x] for x in word], (0, (-len(word))
27         % (T-1)))]
28
29 phash, ptrace = Poseidon().hash(hash_input)
30
31 X = Polynomial.X()
32
33 def next_power_of_two(n: int) -> int:
34     if n <= 0:
35         return 1
36     # If n is already a power of two, return n
37     if (n & (n - 1)) == 0:
38         return n
39     power = 1
40     while power < n:
41         power <<= 1
42     return power
```

Annexe: Code – Prouveur (2/7)

```
1 def hash_input_index(i,j):
2     if (i % (r_f+r_p) == 0 and j < 7 and i//(r_f+r_p) < len(hash_input)//7):
3         return hash_input[7*(i//(r_f+r_p))+j]
4     else:
5         return FieldElement(0)
6
7 ptrace_domain_size = next_power_of_two(len(ptrace))
8 gp = FieldElement.generator() ** (3*2**30/(ptrace_domain_size))
9 x = [gp**i for i in range(len(trace['i']))]
10 z = [gp**i for i in range(len(ptrace))]
11
12 state_polynomial = interpolate_poly(x, [FieldElement(automata.state_map[s] for s in trace['s'])])
13 caracter_polynomial = interpolate_poly(x, [FieldElement(automata.caracter_map[c] for c in trace['c'])])
14 transi_x = [FieldElement(automata.state_map[s]*automata.caracter_map[c] for s,c in tqdm(product(automata
    .states, automata.alphabet)))]
15
16 transition_polynomial = interpolate_poly(transi_x, [FieldElement(automata.state_map[automata.transition[s]
    ][c]) for (s,c) in tqdm(product(automata.states, automata.alphabet))])
17
18 step_polynomial = state_polynomial(gp*X) - transition_polynomial(state_polynomial(X)*caracter_polynomial(
    X))
19
20 ptrace_polynomials = [interpolate_poly(z, [row[i] for row in ptrace]) for i in range(0,len(ptrace[0]))]
21 arc_polys = [interpolate_poly(z, [FieldElement(ARC[j%(r_f+r_p)])[i] for j in range(len(ptrace))] for i
    in range(len(ARC[0]))]
22 hash_input_poly = [interpolate_poly(z, [hash_input_index(i,j) for i in range(len(z))] for j in range(len(
    ptrace[0]))]
23
24 w = FieldElement.generator()
25 h = FieldElement.generator() ** (3*2**30/(ptrace_domain_size*blowup_factor))
26 H = [h**i for i in range(ptrace_domain_size*blowup_factor)]
27 eval_domain = [w*h for h in H]
28
29 state_eval = [state_polynomial(d) for d in tqdm(eval_domain)]
30 caracter_eval = [caracter_polynomial(d) for d in tqdm(eval_domain)]
31 #transition_eval = [transition_polynomial(state*caracter) for state,caracter in tqdm(product(state_eval,
    caracter_eval))]
32 transition_eval = [transition_polynomial(state*caracter) for state,caracter in zip(state_eval,
    caracter_eval)]
33 ptrace_evals = [[p(x) for x in tqdm(eval_domain)] for p in ptrace_polynomials]
34 hash_evals = [[p(x) for x in tqdm(eval_domain)] for p in hash_input_poly]
35 arc_polys_eval = [[p(x) for x in tqdm(eval_domain)] for p in arc_polys]
```

Annexe: Code – Prouveur (3/7)

```
1 state_merkle = MerkleTree(state_eval)
2 character_merkle = MerkleTree(character_eval)
3 transition_merkle = MerkleTree(transition_eval)
4 ptrace_merkle = MerkleTree([e for p in ptrace_evals for e in p])
5 hash_merkle = MerkleTree([e for p in hash_evals for e in p])
6 arc_merkle = MerkleTree([e for p in arc_polys_eval for e in p])
7
8 initial_constraint_p = (state_polynomial - automata.state_map['q0'])/(X-1)
9 Z_G = Polynomial([FieldElement.one()])
10 for i in range(len(x)):
11     Z_G = Z_G * (X-x[i])
12 ap = Polynomial([FieldElement.one()])
13 for c in automata.alphabet:
14     ap = ap * (character_polynomial - automata.character_map[c])
15 character_constraint_p = ap/Z_G
16 ap = Polynomial([FieldElement.one()])
17 for f in automata.accept_states:
18     ap = ap * (state_polynomial - automata.state_map[f])
19 accepting_constraint_p = ap/(X-x[-1])
20 step_constraint_p = step_polynomial/(Z_G/(X-x[-1]))
21 poseidon_full_round_polynomials = mds@[(ptrace_polynomials[j](X) + hash_input_poly[j](X) + arc_polys[j](X)
22     )]**5 for j in range(T)]
23 poseidon_partial_round_polynomials = mds@[(ptrace_polynomials[j](X) + hash_input_poly[j](X) + arc_polys[j]
24     )(X)]**(5 if j == 0 else 1) for j in range(T)]
25 X_G = Polynomial([FieldElement.one()])
26 Z_G = Polynomial([FieldElement.one()])
27 for i in range(len(ptrace)-1):
28     r = i%(r_f+r_p)
29     if(r < r_f/2 or r >= r_f/2+r_p):
30         X_G = X_G * (X-z[i])
31     else:
32         Z_G = Z_G * (X-z[i])
33 poseidon_constraint_full_round = [(ptrace_polynomials[j](gp*X) - poseidon_full_round_polynomials[j](X))/
34     X_G for j in range(len(ptrace_polynomials))]
35 poseidon_constraint_partial_round = [(ptrace_polynomials[j](gp*X) - poseidon_partial_round_polynomials[j]
36     )(X))/Z_G for j in range(len(ptrace_polynomials))]
37 poseidon_constraints_initial = [p/(X-1) for p in ptrace_polynomials]
38 poseidon_output_constraint = (ptrace_polynomials[0](X)-phash[0])/(X-z[-1])
```

Annexe: Code – Prouveur (4/7)

```
1 channel = Channel()
2 channel.send(str(ptrace_domain_size))
3 channel.send(str(len(ptrace)))
4 channel.send(str(len(eval_domain)))
5 channel.send(str(len(trace['i'])))
6 channel.send(str(phash[0]))
7 channel.send(state_merkle.root)
8 channel.send(caracter_merkle.root)
9 channel.send(transition_merkle.root)
10 channel.send(ptrace_merkle.root)
11 channel.send(hash_merkle.root)
12 channel.send(arc_merkle.root)
13
14 ps = [poseidon_output_constraint, initial_constraint_p, caracter_constraint_p, accepting_constraint_p,
      step_constraint_p, *poseidon_constraints_initial, *poseidon_constraint_full_round, *
      poseidon_constraint_partial_round]
15 alphas = [channel.receive_random_field_element() for _ in range(len(ps))]
16 cp = reduce(lambda x, y: x+y, map(lambda a, p: a*p, alphas, ps), FieldElement(0))
17
18 cp_eval = [cp(x) for x in eval_domain]
19 cp_merkle = MerkleTree(cp_eval)
20 channel.send(cp_merkle.root)
21
22 def fold_domain(domain):
23     return [x**2 for x in domain[:len(domain)//2]]
24
25 def fold(p, beta):
26     odd = p.poly[1::2]
27     even = p.poly[::2]
28     o = Polynomial(odd)
29     e = Polynomial(even)
30     return e + beta*o
31
32 def next_fri_layer(poly, domain, beta):
33     next_poly = fold(poly, beta)
34     next_domain = fold_domain(domain)
35     next_layer = [next_poly(x) for x in next_domain]
36     return next_poly, next_domain, next_layer
37
38 cp.degree(), len(cp_eval)
```

Annexe: Code – Prouveur (5/7)

```
1 def FriCommit(cp, domain, cp_eval, cp_merkle, channel):
2     fri_polys = [cp]
3     fri_domains = [domain]
4     fri_layers = [cp_eval]
5     fri_merkles = [cp_merkle]
6     betas = []
7     i = 0
8     while fri_polys[-1].degree() > 0:
9         beta = channel.receive_random_field_element()
10        betas.append(beta)
11        next_poly, next_domain, next_layer = next_fri_layer(fri_polys[-1], fri_domains[-1], beta)
12        fri_polys.append(next_poly)
13        fri_domains.append(next_domain)
14        fri_layers.append(next_layer)
15        fri_merkles.append(MerkleTree(next_layer))
16        channel.send(fri_merkles[-1].root)
17        i+=1
18    channel.send("FINISHED_FRI")
19    channel.send(str(fri_polys[-1].poly[0]))
20    return fri_polys, fri_domains, fri_layers, fri_merkles, betas
21
22 fri_polys, fri_domains, fri_layers, fri_merkles, betas = FriCommit(cp, eval_domain, cp_eval, cp_merkle,
23    channel)
24
25 len(fri_layers[-1])
26
27 def decommit_on_fri_layers(idx, channel):
28     prev_idx = None
29     prev_sibidx = None
30     p_idx = None
31     i = 0
32     for layer, merkle in zip(fri_layers[:-1], fri_merkles[:-1]):
33         length = len(layer)
34         idx = idx % length
35         sib_idx = (idx + length // 2) % length
36         channel.send(str(layer[idx]))
37         channel.send(str(merkle.get_authentication_path(idx)))
38         channel.send(str(layer[sib_idx]))
39         channel.send(str(merkle.get_authentication_path(sib_idx)))
40     channel.send(str(fri_layers[-1][0]))
```

Annexe: Code – Prouveur (6/7)

```
1 def decommit_on_query(idx, channel):
2     assert idx + blowup_factor < len(eval_domain), f'query index: {idx} is out of range. Length of layer:
        {len(eval_domain)}.'
3     channel.send(str(state_eval[idx%len(eval_domain)])) # f(x).
4     channel.send(str(state_merkle.get_authentication_path(idx%len(eval_domain)))) # auth path for f(x).
5     assert(state_eval[(idx + blowup_factor)%len(eval_domain)] == state_polynomial(gp*eval_domain[idx%len(
        eval_domain)]))
6     channel.send(str(state_eval[(idx + blowup_factor)%len(eval_domain)])) # f(gx).
7     channel.send(str(state_merkle.get_authentication_path((idx + blowup_factor)%len(eval_domain)))) #
        auth path for f(gx).
8     assert (idx + blowup_factor)%len(eval_domain) < len(caracter_eval), f'query index: {idx%len(
        eval_domain)} is out of range. Length of layer: {len(caracter_eval)}.'
9     channel.send(str(caracter_eval[idx%len(eval_domain)])) # f(x).
10    channel.send(str(caracter_merkle.get_authentication_path(idx%len(eval_domain)))) # auth path for f(x)
11
12    assert (idx + blowup_factor)%len(eval_domain) < len(transition_eval), f'query index: {idx%len(
        eval_domain)} is out of range. Length of layer: {len(transition_eval)}.'
13    transi_x = transition_polynomial(caracter_eval[idx%len(eval_domain)]*state_eval[idx%len(eval_domain)
        ])
14    channel.send(str(transi_x)) # f(x).
15    transi_idx = (len(eval_domain)*idx+idx)%len(eval_domain)
16    channel.send(str(transition_merkle.get_authentication_path(transi_idx))) # auth path for f(x).
17
18    #### POSEIDON ####
19    for row in range(len(ptrace[0])):
20        trace_x = ptrace_evals[row][idx%len(eval_domain)]
```

Annexe: Code – Prouveur (7/7)

```
1 channel.send(str(trace_x))
2 channel.send(str(ptrace_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain)
3 )))))
4 assert(ptrace_polynomials[row][eval_domain[idx%len(eval_domain)]] == trace_x)
5
6 next_trace_x = ptrace_evals[row][(idx+blowup_factor)%len(eval_domain)]
7 channel.send(str(next_trace_x))
8 channel.send(str(ptrace_merkle.get_authentication_path((len(eval_domain)*row+((idx+blowup_factor)
9 %len(eval_domain)))))
10 assert(ptrace_polynomials[row][gp*eval_domain[idx%len(eval_domain)]] == next_trace_x)
11
12 hash_x = hash_evals[row][idx%len(eval_domain)]
13 channel.send(str(hash_x))
14 channel.send(str(hash_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain))
15 ))))
16
17 arc_x = arc_polys_eval[row][idx%len(eval_domain)]
18 channel.send(str(arc_x))
19 channel.send(str(arc_merkle.get_authentication_path((len(eval_domain)*row+(idx%len(eval_domain))
20 ))))
21
22 decommit_on_fri_layers(idx, channel)
23
24 def decommit_fri(channel):
25     for query in range(N_QUERY):
26         decommit_on_query(channel.receive_random_int(0, len(eval_domain)), channel)
27
28 decommit_fri(channel)
29
30 proof_file = open("proof.txt", "w")
31 proof_file.write(str(channel.proof))
32
33 print_prover_success_banner(start_all, channel, len(ptrace), len(state_eval))
```

Annexe: Code – Vérifieur (1/7)

```
1 import ast
2 import constants
3 from constants import blowup_factor
4 from channel import Channel
5 from field import FieldElement
6 from merkle import verify_decommitment
7 from poseidon import generate_cauchy_matrix
8 from funtools import reduce
9 from utils import Automata

10
11 proof_file = open("proof.txt", "r")
12 proof = ast.literal_eval(proof_file.read())
13 automata = Automata.from_json(open("automata.json", "r").read())
14
15 def parse_proof(method, parse = None):
16     global v_current_channel_idx
17     global v_proof
18     #print("channel", proof[current_channel_idx], "method", method)
19     v_current_channel_idx+=1
20     if parse == None:
21         return v_proof[v_current_channel_idx-1][len(method)+1:]
22     else:
23         return parse(v_proof[v_current_channel_idx-1][len(method)+1:])
24
25 channel = Channel(proof)
26 v_proof = channel.proof
27 v_c = Channel()
28 v_current_channel_idx = 0
29 PTRACE_DOMAIN_SIZE=parse_proof("send", lambda x: int(x))
30 POSEIDON_TRACE_EVAL_LENGTH=parse_proof("send", lambda x: int(x))
31 EVAL_DOMAIN_LENGTH=parse_proof("send", lambda x: int(x))
32 AUTOMATA_TRACE_LENGTH=parse_proof("send", lambda x: int(x))
33 v_hash = v_proof[v_current_channel_idx][5:]
34 v_c.send(v_hash)
35 v_current_channel_idx+=1
36 v_state_merkle = v_proof[v_current_channel_idx][5:]
37 v_c.send(v_state_merkle)
```


Annexe: Code – Vérifieur (2/7)

```
1  v_current_channel_idx+=1
2  v_caracter_merkle = v_proof[v_current_channel_idx][5:]
3  v_c.send(v_caracter_merkle)
4  v_current_channel_idx+=1
5  v_transition_merkle = v_proof[v_current_channel_idx][5:]
6  v_c.send(v_transition_merkle)
7  v_current_channel_idx+=1
8  v_ptrace_merkle = v_proof[v_current_channel_idx][5:]
9  v_c.send(v_ptrace_merkle)
10 v_current_channel_idx+=1
11 v_hash_merkle = v_proof[v_current_channel_idx][5:]
12 v_c.send(v_hash_merkle)
13 v_current_channel_idx+=1
14 v_arc_merkle = v_proof[v_current_channel_idx][5:]
15 v_c.send(v_arc_merkle)
16 v_current_channel_idx+=1
17 v_alphas = []
18 for i in range(constants.CONSTRAINTS_LENGTH):
19     v_a = FieldElement(int(v_proof[v_current_channel_idx][len('receive_random_field_element:')]))
20     v_current_channel_idx+=1
21     v_alphas.append(v_a)
22 #a0 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
23 #assert(c.receive_random_field_element() == a0)
24 #current_channel_idx+=1
25 #a1 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
26 #assert(c.receive_random_field_element() == a1)
27 #current_channel_idx+=1
28 #a2 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
29 #assert(c.receive_random_field_element() == a2)
30 #current_channel_idx+=1
31 #a3 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
32 #assert(c.receive_random_field_element() == a3)
33 #current_channel_idx+=1
34 #a4 = FieldElement(int(proof[current_channel_idx][len('receive_random_field_element:')]))
35 #assert(c.receive_random_field_element() == a4)
36 #current_channel_idx+=1
37 cp_merkle_root = v_proof[v_current_channel_idx][len('send:'):]
38 v_current_channel_idx+=1
```

Annexe: Code – Vérifieur (3/7)

```
1
2 w = FieldElement.generator()
3 h = FieldElement.generator() ** (3*2**30/(PTRACE_DOMAIN_SIZE*blowup_factor))
4 gp = FieldElement.generator() ** (3*2**30/(PTRACE_DOMAIN_SIZE))
5
6 #def x(i):
7 #    return gp**i
8
9 def x(i):
10     return gp**(i%AUTOMATA_TRACE_LENGTH)
11
12 def z(i):
13     return gp**(i%POSEIDON_TRACE_EVAL_LENGTH)
14
15 #x = [gp**i for i in range(AUTOMATA_TRACE_LENGTH)]
16 #z = [gp**i for i in range(POSEIDON_TRACE_EVAL_LENGTH)]
17 mds = generate_cauchy_matrix(constants.T)
18
19 def verify_FRI_commitment():
20     global v_proof
21     global v_current_channel_idx
22     v_betas = []
23     v_roots = [cp_merkle_root]
24     v_l = 0
25     while v_proof[v_current_channel_idx] != "send:FINISHED_FRI":
26         v_betas.append(FieldElement(int(v_proof[v_current_channel_idx][len('receive_random_field_element:
27             '):])))
28         v_current_channel_idx+=1
29         v_roots.append(v_proof[v_current_channel_idx][len('send:')])
30         v_current_channel_idx+=1
31         v_l+=1
32     v_current_channel_idx+=1
33     v_poly0 = int(v_proof[v_current_channel_idx][len('send:')])
34     v_current_channel_idx+=1
35     return v_betas, v_roots, v_poly0, v_l
36
37 v_betas, v_roots, v_poly0, v_l = verify_FRI_commitment()
38
39 print(v_l, 2**10)
```

Annexe: Code – Vérifieur (4/7)

```
1
2 def verify_query_decommitment(l):
3     global current_channel_idx
4     for query in range(constants.N_QUERY):
5         # r = int(proof[current_channel_idx][len('receive_random_field_element:')])
6         r = parse_proof("receive_random_int", int)
7         # current_channel_idx+=1
8         state_x = parse_proof("send", lambda x: FieldElement(int(x)))
9         #state_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
10        #current_channel_idx+=1
11        state_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
12        state_x2 = parse_proof("send", lambda x: FieldElement(int(x)))
13        #state_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
14        #current_channel_idx+=1
15        state_x2_path = parse_proof("send", lambda x: ast.literal_eval(x))
16        #state_x_path = ast.literal_eval(proof[current_channel_idx][len('send:')]))
17        #current_channel_idx+=1
18        assert(verify_decommitment(r, state_x, state_x_path, v_state_merkle))
19        assert(verify_decommitment((r+blowup_factor), state_x2, state_x2_path, v_state_merkle))
20
21        #caracter_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
22        #current_channel_idx+=1
23        caracter_x = parse_proof("send", lambda x: FieldElement(int(x)))
24        caracter_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
25        #caracter_x_path = proof[current_channel_idx][len('send:')])
26        #current_channel_idx+=1
27        assert(verify_decommitment(r, caracter_x, caracter_x_path, v_caracter_merkle))
28
29        transition_x = parse_proof("send", lambda x: FieldElement(int(x)))
30        transition_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
31        assert(verify_decommitment((EVAL_DOMAIN_LENGTH*r+r), transition_x, transition_x_path,
32                                     v_transition_merkle))
33
34        poseidon_x = []
35        poseidon_path = []
36        poseidon_next_x = []
37        poseidon_next_path = []
38        hash_x = []
39        hash_path = []
```

Annexe: Code – Vérifieur (5/7)

```
1 arc_x = []
2 arc_path = []
3
4 for i in range(constants.T):
5     poseidon_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
6     poseidon_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
7     assert(verify_decommitment((i*EVAL_DOMAIN_LENGTH + (r)),poseidon_x[-1],poseidon_path[-1],
8         v_ptrace_merkle))
9
10    poseidon_next_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
11    poseidon_next_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
12    assert(verify_decommitment((i*EVAL_DOMAIN_LENGTH + ((r + blowup_factor))),poseidon_next_x
13        [-1],poseidon_next_path[-1],v_ptrace_merkle))
14
15    hash_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
16    hash_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
17    assert(verify_decommitment((i*EVAL_DOMAIN_LENGTH + (r)),hash_x[-1],hash_path[-1],
18        v_hash_merkle))
19
20    arc_x.append(parse_proof("send", lambda x: FieldElement(int(x))))
21    arc_path.append(parse_proof("send", lambda x: ast.literal_eval(x)))
22    assert(verify_decommitment((i*EVAL_DOMAIN_LENGTH + (r)),arc_x[-1],arc_path[-1],v_arc_merkle)
23        )
24
25    #transition_x = FieldElement(int(proof[current_channel_idx][len('send:')]))
26    #current_channel_idx+=1
27    #transition_x_path = proof[current_channel_idx][len('send:'):]
28    #current_channel_idx+=1
29
30    layers = []
31    layer_paths = []
32
33    #length = 2**((v_l+1)
34    length = EVAL_DOMAIN_LENGTH
35    cps = []
36
37    for j in range(0,1):
38        idx = r % length
39        sib_idx = (r + length // 2) % length
40        layer_x = parse_proof("send", lambda x: FieldElement(int(x)))
41        #layer_x = int(proof[current_channel_idx][len('send:')])
42        #current_channel_idx+=1
```

Annexe: Code – Vérifieur (6/7)

```
1 layer_x_path = parse_proof("send", lambda x: ast.literal_eval(x))
2 assert(verify_decommitment(idx, layer_x, layer_x_path, v_roots[j]))
3 #layer_x_path = proof[current_channel_idx][len('send:')]
4 #current_channel_idx+=1
5 layer_sibx = parse_proof("send", lambda x: FieldElement(int(x)))
6 #layer_sibx = int(proof[current_channel_idx][len('send:')])
7 #current_channel_idx+=1
8 #layer_sibx_path = proof[current_channel_idx][len('send:')]
9 layer_sibx_path = parse_proof("send", lambda x: ast.literal_eval(x))
10 assert(verify_decommitment(sib_idx, layer_sibx, layer_sibx_path, v_roots[j]))
11 e = (w*h*idx)**(2*j)
12
13 g_x = (layer_x + layer_sibx)/FieldElement(2)
14 h_x = (layer_x - layer_sibx)/(2*e)
15 cp_ip1 = g_x + v_betas[j]*h_x
16 cps.append(cp_ip1)
17
18 if j == 0:
19     initial_constraint = (state_x-automata.state_map['q0'])/(e-1)
20     Z_G = FieldElement(1)
21     for i in range(AUTOMATA_TRACE_LENGTH):
22         Z_G = Z_G * (e-x(i))
23     ap = FieldElement(1)
24     for c in automata.alphabet:
25         ap = ap * (character_x - automata.caracter_map[c])
26     caracter_constraint = ap/Z_G
27     ap = FieldElement(1)
28     for f in automata.accept_states:
29         ap = ap * (state_x - automata.state_map[f])
30     accepting_constraint = ap/(e-x(-1))
31     step_constraint = (state_x2 - transition_x)/(Z_G/(e-x(-1)))
32     X_G = FieldElement(1)
33     Y_G = FieldElement(1)
34     for i in range(POSEIDON_TRACE_EVAL_LENGTH-1):
35         round_x = i%(constants.r_f+constants.r_p)
36         if(round_x < constants.r_f/2 or round_x >= constants.r_f/2+constants.r_p):
37             X_G = X_G * (e-z(i))
38     else:
```

Annexe: Code – Vérifieur (7/7)

```
1         Y_G = Y_G * (e-z(i))
2         v_poseidon_full_round = mds@[(poseidon_x[j] + hash_x[j] + arc_x[j])**5 for j in range(
3             constants.T)]
4         v_poseidon_partial_round = mds@[(poseidon_x[j] + hash_x[j] + arc_x[j])**5 if j == 0 else
5             1) for j in range(constants.T)]
6         poseidon_full_round_cr = [(poseidon_next_x[j] - v_poseidon_full_round[j])/X_G for j in
7             range(constants.T)]
8         poseidon_partial_round_cr = [(poseidon_next_x[j] - v_poseidon_partial_round[j])/Y_G for j
9             in range(constants.T)]
10        poseidon_initial_cr = [poseidon_x[j]/(e-1) for j in range(constants.T)]
11        poseidon_output_cr = (poseidon_x[0]-FieldElement(int(v_hash)))/(e-z(-1))
12
13        v_ps = [poseidon_output_cr, initial_constraint, character_constraint, accepting_constraint,
14            step_constraint, *poseidon_initial_cr, *poseidon_full_round_cr, *
15            poseidon_partial_round_cr]
16        cp_e = reduce(lambda x, y: x+y, map(lambda a, p: a*p, v_alphas, v_ps), FieldElement(0))
17        assert(cp_e == layer_x)
18    else:
19        assert(cps[j-1] == layer_x)
20
21    layers.append((layer_x, layer_sibx))
22    layer_paths.append((layer_x_path, layer_sibx_path))
23    length //= 2
24
25    last_layer_x = parse_proof("send", int)
26    assert(cps[-1] == last_layer_x)
27
28    verify_query_decommitment(v_1)
29
30    print_success_banner(verification_start, channel, EVAL_DOMAIN_LENGTH, POSEIDON_TRACE_EVAL_LENGTH)
```